

MyCS — 多人联机波次生存射击游戏

基于 Unreal Engine 5.7 First Person 模板 · 纯 C++ 实现 · 2026.06

目录

- [1. 项目概述](#)
- [2. 需求完成度评估](#)
- [3. 整体架构](#)
- [4. 模块详解](#)
- [5. 网络同步方案](#)
- [6. 核心数据流](#)
- [7. 关键技术点](#)
- [8. 操作说明](#)
- [9. 游戏规则](#)
- [10. 总结与扩展](#)

一、项目概述

项目	说明
项目名称	MyCS — Multiplayer Wave Survival Shooter
GitHub	https://github.com/s1mple543/UE-DEMO.git
引擎版本	Unreal Engine 5.7
开发语言	C++ (纯代码, 零蓝图逻辑)
项目规模	50+ 源文件, 约 6000 行 C++ 代码
开发周期	约 2 周

核心玩法	4 人联机合作，每 15 秒生成 1 只敌人，总计 20 只，全清获胜
同步方案	服务端权威式状态同步

二、需求完成度评估

要求 1：会移动和攻击玩家的敌人，玩家可以击败敌人

✅ 完成

功能	实现文件	说明
NPC 移动/寻路	ShooterNPC + ShooterAIController	StateTree 驱动 AI，导航网格寻路
NPC 攻击玩家	ShooterNPC::StartShooting()	AI 感知系统检测玩家，锁定射击
玩家击败敌人	ShooterCharacter::TakeDamage()	投射物命中造成伤害，HP 归零敌人死亡
武器系统	ShooterWeapon / ShooterProjectile / ShooterPickup	手枪/步枪/榴弹发射器，弹药消耗
手动换弹	ShooterWeapon::StartReload()	R 键换弹，换弹期间不能射击

要求 2：得分和游戏胜利机制

✅ 完成

机制	实现	细节
游戏规则	ShooterGameMode	每 15 秒生成 1 只敌人，总计 20 只
胜利条件	HandleVictory()	消灭全部 20 只敌人
失败条件	HandleDefeat()	所有玩家命数耗尽
命数系统	MultiShooterPlayerState	每人 3 条命，死亡后消耗 1 条

击杀计分	AddKill() / AddDeath()	每杀一只 +100 分
血量恢复	OnHealthRegen()	每 10 秒恢复 50HP，上限 500

要求 3：多人网络对战

✅ 完成

功能	实现	细节
对象复制	bReplicates = true	Character/NPC/Weapon/Projectile 均支持复制
服务器 RPC	ServerStartFiring() / ServerReload()	客户端射击/换弹请求发送到服务器
属性同步	DOREPLIFETIME	HP、弹药数、换弹状态、击杀数等属性网络同步
游戏状态	MultiShooterGameState	总敌人数、存活敌人数、游戏结束标志
玩家状态	MultiShooterPlayerState	击杀数、死亡数、剩余命数
联机模式	OnlineSubsystemNull	LAN 联机，支持 4 人同玩
角色碰撞	ShooterProjectile	友军伤害开启，玩家可互伤

三、整体架构

Source/MyCS/	
└── MyCS/	# 基础层（FPS 模板基类）
│ └── MyCSCharacter.h/.cpp	# 角色基础（移动/跳跃/相机）
│ └── MyCSPlayerController	# 控制器基础
│ └── MyCSGameMode	# 游戏模式基础
│ └── MyCSCameraManager	# 相机约束
│	
└── Variant_Shooter/	# 玩法层（核心游戏逻辑）

4.2 武器系统 (ShooterWeapon / ShooterProjectile / ShooterPickup)

ShooterWeapon 武器基类

- 双网格渲染 (第一人称 + 第三人称)
- 弹药系统: MagazineSize 弹匣容量, CurrentBullets 当前弹药
- 全自动/半自动射击模式 (bFullAuto + RefireRate)
- 弹道散布 + 后坐力
- AI 感知噪声: MakeNoise()

换弹系统 (我们新增) :

```
void StartReload()      // 开始换弹, 标记 bIsReloading, 设置换弹计时器
void FinishReload()     // 换弹完成, 填满弹匣, 更新 HUD
```

关键设计: 使用独立的 ReloadTimer , 与射击的 RefireTimer 分离, 避免定时器冲突。

ShooterProjectile 投射物

- SphereComponent 碰撞 + ProjectileMovementComponent 弹道
- HitDamage 伤害值, 通过 ApplyDamage() 统一接口
- 爆炸模式: bExplodeOnHit 开启范围伤害

友军伤害规则:

```
// 玩家 → NPC: 造成伤害
// NPC → 玩家: 造成伤害 (敌人打玩家)
// 玩家 → 玩家: 造成伤害 (友军伤害开启)
// NPC → NPC: 跳过伤害 (防止内讧)
```

ShooterPickup 武器拾取

- 基于 DataTable 配置武器类型 (Pistol/Rifle/GrenadeLauncher)
- 碰撞重叠触发 AddWeaponClass()
- 已持有武器时自动补满弹匣 + 切换到该武器

4.3 玩家角色 (ShooterCharacter)

继承自 MyCSCharacter , 实现 IShooterWeaponHolder 接口。

状态	类型	同步方式
HP	float CurrentHP	ReplicatedUsing=OnRep_CurrentHP
最大血量	float MaxHP = 500	仅服务器
瞄准状态	bool bIsAiming	本地

关键功能实现：

伤害与死亡：

```
float TakeDamage(float Damage, ...) {
    CurrentHP -= Damage;
    if (CurrentHP <= 0) Die();
    OnDamaged.Broadcast(HP/MaxHP); // 更新 HUD
}
```

射击网络同步：

```
void DoStartFiring() {
    if (HasAuthority()) CurrentWeapon->StartFiring();
    else ServerStartFiring(); // RPC → 服务器
}
```

血量恢复：

```
void OnHealthRegen() {
    CurrentHP = FMath::Min(MaxHP, CurrentHP + 50.0f);
    // 每 10 秒触发，仅服务器执行
}
```

瞄准系统 (ADS) :

```
// Tick() 中平滑插值 FOV
float Target = bIsAiming ? AimFOV(35°) : DefaultFOV(70°);
float NewFOV = FMath::FInterpTo(Current, Target, Delta, 12.0f);
```

4.4 敌人 AI (ShooterNPC + ShooterAIController)

AI 架构： StateTree 行为树

```
SenseEnemies → 检测玩家 (AI Perception 视觉+听觉)
    ↓ 发现敌人
FaceActor → 面向目标
ShootAtTarget → 开火
    ↓ 丢失目标
Roam → 随机巡逻
```

击杀逻辑：

```
void Die() {
    OnPawnDeath.Broadcast(); // 触发 GameMode 击杀计数
    if (LastDamageInstigator) // 只记录玩家控制的击杀者
        PS->AddKill();
    GetMesh()->SetSimulatePhysics(true); // 布娃娃物理
}
```

4.5 游戏模式 (ShooterGameMode)

游戏规则的核心控制器，负责刷怪和胜负判定。

关键数据结构：

```
int32 TotalSpawned = 0;    // 已生成数量
int32 TotalKilled = 0;    // 已击杀数量
bool bFinishedSpawning;   // 是否已生成完毕
bool bGameOver;           // 游戏是否结束
```

胜负判定逻辑：

```
// 胜利：所有敌人生成完毕且全部击杀
if (bFinishedSpawning && TotalKilled >= MaxEnemies)
    HandleVictory();

// 失败：所有玩家命数为 0
void CheckDefeat() {
    for (PS : GameState->PlayerArray)
        if (PS->RemainingLives > 0) return;
    HandleDefeat();
}
```

4.6 HUD 与菜单系统 (MultiShooterGameHUD)

纯 C++ AHUD 子类，使用 Canvas 直接绘制，零蓝图依赖。

UI 元素	位置	技术实现
敌人信息	左上角	Canvas DrawText + 背景 Tile
玩家统计	左上角	击杀数/分数/命数/死亡数
弹药计数	右下角	12/30 或 RELOADING...
空弹提示	弹药下方	红色 [R] RELOAD
血条	右下角	绿/橙/红三段渐变
十字准星	屏幕中心	4 条白色线段 + 中心点
瞄准镜	全屏	程序化 UTexture2D 纹理
主菜单	全屏	F10 呼出, Host/Join/Quit
游戏结束	全屏	半透明遮罩 + 结果文字

瞄准镜技术：在 `BeginPlay()` 中动态生成一张 1024×1024 的纹理，包含全黑背景 + 透明圆形镜片 + 绿色十字准线 + 边缘暗环。开镜时一次 `DrawTexture()` 调用完成渲染，性能零开销。

菜单系统：F10 呼出/关闭，游戏暂停输入。支持鼠标点击和键盘 Enter 确认。

五、网络同步方案

5.1 同步架构选择

本项目采用 **服务端权威式状态同步** (Server-Authoritative State Synchronization)，这是 FPS 游戏的行业标准方案，与《堡垒之夜》《使命召唤》《绝地求生》采用相同的底层机制。

对比项	状态同步 (本项目)	帧同步 (Lockstep)
同步内容	游戏状态 (位置/HP/分数)	玩家输入 (按下按键)

防作弊	服务器权威，难作弊	需额外校验
客户端预测	支持	不支持
典型应用	FPS / MMO / ARPG	RTS / 格斗游戏
UE5 实现	Actor Replication + RPC	Input Replication

5.2 属性复制 (DOREPLIFETIME)

MultiShooterGameState (广播给所有客户端) :

```
DOREPLIFETIME(GameState, MaxEnemies);
DOREPLIFETIME(GameState, TotalSpawned);
DOREPLIFETIME(GameState, TotalKilled);
DOREPLIFETIME(GameState, EnemiesRemaining);
DOREPLIFETIME(GameState, bGameOver);
DOREPLIFETIME(GameState, bVictory);
```

MultiShooterPlayerState (每个玩家看到自己的数据) :

```
DOREPLIFETIME(PlayerState, Kills);
DOREPLIFETIME(PlayerState, Deaths);
DOREPLIFETIME(PlayerState, RemainingLives);
```

ShooterWeapon (武器状态同步到所有客户端) :

```
UPROPERTY(Replicated) int32 CurrentBullets;
UPROPERTY(Replicated) bool bIsReloading;
DOREPLIFETIME(ShooterWeapon, CurrentBullets);
DOREPLIFETIME(ShooterWeapon, bIsReloading);
```

5.3 远程过程调用 (RPC)

RPC	方向	用途
ServerStartFiring()	客户端 → 服务器	请求开始开枪
ServerStopFiring()	客户端 → 服务器	请求停止开枪

ServerReload()	客户端 → 服务器	请求换弹
----------------	-----------	------

5.4 RepNotify 同步

HP 属性使用 `ReplicatedUsing = OnRep_CurrentHP`，服务器修改 HP 后自动触发客户端的 `OnRep_CurrentHP` 回调，广播 HP 变化事件更新 HUD。

5.5 网络配置

```
OnlineSubsystemNull      // LAN 联机（无需 Steam）
MaxPlayers=4             // 最大 4 人
open Lvl_Shooter?listen  // 主机命令
open IP:7777             // 加入命令
```

六、核心数据流

6.1 射击流程

```
玩家点击左键
→ DoStartFiring() [客户端]
  → ServerStartFiring() [RPC → 服务器]
    → Weapon->StartFiring()
      → Fire() → FireProjectile()
        → SpawnActor<AShooterProjectile>
          → NotifyHit() → ProcessHit()
            → ApplyDamage() → TakeDamage()
```

6.2 刷怪流程

```
GameMode::BeginPlay()
→ 5 秒后开始刷怪
  → 每 15 秒生成 1 只
    → NPC 绑定 OnPawnDeath → OnEnemyKilled
      → 20 只生成完毕 → bFinishedSpawning = true
        → NPC 死亡 → TotalKilled++
          → 若 TotalKilled >= 20 → Victory
```

6.3 网络同步流程



七、关键技术点

7.1 换弹系统的定时器隔离

换弹使用独立于射击的 `ReloadTimer`，两个 `FTimerHandle` 互不干扰，解决了早期版本中换弹和射击定时器冲突导致换弹卡死的问题。

7.2 NPC 自相残杀防护

在 `ShooterProjectile::ProcessHit()` 中检测：如果射击者和目标都不是玩家控制，跳过伤害计算。同时在 `ShooterNPC::TakeDamage()` 中只记录 `IsPlayerControlled()` 的击杀者，消除虚假击杀计数。

7.3 程序化瞄准镜纹理

在 C++ 中使用 `UTexture2D::CreateTransient()` 动态生成 1024×1024 纹理，通过像素操作绘制黑色遮罩 + 透明圆形镜片 + 绿色十字线。单次 `DrawTexture()` 调用完成渲染，比逐行扫描的方式更高效、更美观。

7.4 Canvas 纯 C++ HUD

完整 HUD 系统使用 UE5 的 `AHUD::DrawHUD()` + `FCanvasItem` 绘制，包括准星、血条、弹药、敌人信息、主菜单、游戏结束遮罩，全部不依赖任何 Blueprint 或 UMG Widget。

八、操作说明

操作	键位	说明
----	----	----

移动	WASD	前后左右
跳跃	空格	
视角	鼠标移动	
射击	鼠标左键	自动/半自动取决于武器
瞄准	鼠标右键	FOV 缩放 + 瞄准镜
换弹	R	手动换弹
切枪	Q 或滚轮	已持有武器间切换
主菜单	F10	Host Game / Join Game / Quit
联机（主机）	F10 → Host	创建房间
联机（加入）	F10 → Join	Localhost 或 LAN

九、游戏规则

规则	详情
敌人刷新	每 15 秒生成 1 只，总数 20 只
胜利条件	消灭全部 20 只敌人
失败条件	所有玩家命数耗尽
玩家命数	每人 3 条命，死亡扣 1 条，归零淘汰
玩家血量	500 HP，每 10 秒自动恢复 50
武器获取	地图上的 Weapon Pickup 拾取
友军伤害	开启，玩家可攻击玩家
NPC 互伤	关闭，NPC 子弹不伤害 NPC

十、总结与扩展方向

项目总结

本项目从 UE5 First Person 模板出发，在两周内完成了一个完整的多人联机波次生存射击游戏。核心技术栈包括：

- **C++ 优先开发**：全部游戏逻辑、AI 行为、网络同步、UI 绘制均在 C++ 中实现
- **服务端权威架构**：所有关键逻辑由服务器执行，状态通过 UE5 复制系统同步
- **StateTree AI**：基于状态树的 AI 编排，简洁可扩展
- **Canvas HUD**：零依赖的纯 C++ UI 系统

可扩展方向

方向	说明
更多武器类型	霰弹枪、狙击枪、近战武器
道具系统	手雷、医疗包、护甲
地图扩展	更多关卡、可破坏环境
AI 升级	Boss 战、不同敌人类型
Steam 联机	集成 OnlineSubsystemSteam
打包发布	PackageGame.bat 脚本一键打包